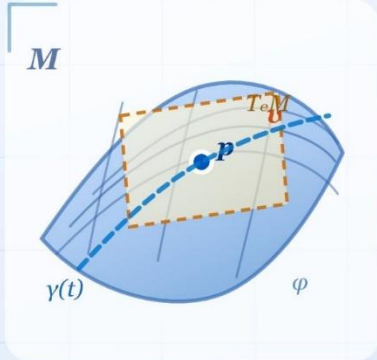


EmbedSim Framework

Technical Architecture & Algorithm Reference

Version 3.1.0 | Open-Source Python / C Control-Systems Simulation
Targets: Infineon AURIX TriCore • ARM Cortex-M4 • FMI 2.0 Co-Simulation



ESF v2.0

EmbedSim

EMBEDDED CONTROL · OPEN SOURCE

Built by Engineers for Engineers

Abstract

EmbedSim is an open-source, Python-native block-diagram simulation framework designed as an affordable, engineering-grade alternative to MATLAB/Simulink for embedded control-systems development. The framework unifies a pure-Python simulation path with a compiled Cython/C execution backend, enabling control algorithms to be written once in C and validated in Python before deployment to 32-bit MCU targets such as the Infineon AURIX TriCore TC3xx and ARM Cortex-M4 families.

This document is a formal technical reference. It describes the mathematical foundations, computational graph model, dependency-resolution algorithms (depth-first search with loop-breaker semantics), ODE integration strategies (Forward Euler, Runge-Kutta 4), dual-backend execution architecture, and the C code-generation pipeline. The treatment is intentionally rigorous and is intended for engineers and researchers who need a precise understanding of the system's internals.

1 Introduction

Modern embedded control design demands tools that span the full V-model: from high-fidelity numerical simulation, through rapid-prototyping on hardware-in-the-loop rigs, to production-quality C code that satisfies MISRA C:2012 and IEC 26262 ASIL-D requirements. Commercial tools address this need at considerable licence cost. EmbedSim addresses the same workflow in the open-source ecosystem.

The framework is structured around three orthogonal concerns:

- Graph model: a directed, acyclic (after loop breaking) block diagram whose nodes are computational units and whose edges carry typed vector signals.
- Execution semantics: a topologically sorted forward-evaluation pass at every discrete timestep, with algebraic feedback loops resolved by causal delay elements.
- Numerical integration: a choice of first-order (Euler) or fourth-order (RK4) ODE solvers applied to all dynamic (stateful) nodes simultaneously.

Section 2 establishes notation and the formal graph model. Section 3 derives the dependency-resolution algorithm. Section 4 covers ODE integration. Section 5 describes the dual-backend architecture and C code generation. Section 6 addresses the feedback loop-breaking mechanism. Section 7 documents the signal type system.

2 Formal Graph Model

2.1 Block Diagram as a Directed Graph

A simulation is modelled as a directed graph $G = (V, E)$ where each vertex $v \in V$ is a VectorBlock instance and each directed edge $(u, v) \in E$ represents a signal flowing from the output port of u to one input port of v . Edges are stored implicitly: each block maintains an ordered list $v.inputs$ of its upstream predecessors.

Formally, the output of block v at time t is a function of its input signals and, for dynamic blocks, an internal state vector:

Static block:	$y_v(t)$	$= f_v(y_{\{u1\}}(t), y_{\{u2\}}(t), \dots)$
Dynamic block:	$y_v(t)$	$= g_v(x_v(t), y_{\{u1\}}(t), \dots)$
	dx_v/dt	$= h_v(x_v(t), y_{\{u1\}}(t), \dots)$

The distinction between static and dynamic nodes is encoded in the boolean attribute `VectorBlock.is_dynamic`. Only dynamic blocks participate in the ODE integration step.

2.2 Signal Type: VectorSignal

Every edge carries a `VectorSignal`, a thin wrapper around a NumPy ndarray. The framework defaults to float32 (single-precision IEEE 754) for MCU compatibility, but individual blocks may request float64 via the dtype constructor argument.

Attribute	Description
value	np.ndarray — the signal vector (float32 by default)
name	Optional string label for plotting and debugging
shape	Derived from value.shape; scalar signals have shape (1,)

Signal dimensionality is not globally enforced at connection time; dimension mismatches are caught at compute time when NumPy operations raise shape errors. This design allows blocks such as `VectorGain` to support both scalar and matrix modes without a separate type hierarchy.

2.3 Block Categories

The framework recognises four functional categories of block:

Category	Description & Examples
Source	No inputs; generate signals as functions of time only. <code>VectorStep</code> , <code>VectorConstant</code> , <code>VectorSine</code> , <code>GaussianNoiseBlock</code> .
Static processing	One or more inputs, no internal state. Output depends only on current inputs. <code>VectorGain</code> , <code>VectorSum</code> , <code>VectorSaturation</code> , <code>VectorProduct</code> .
Dynamic	One or more inputs, internal state vector $x(t)$. Output depends on inputs and state. <code>VectorIntegrator</code> , <code>StateSpaceBlock</code> , <code>TransferFunctionBlock</code> , <code>SpeedPIBlock</code> .
Sink	No downstream consumers; record outputs. <code>VectorEnd</code> captures history. Also <code>VectorScope</code> for arbitrary blocks.

Loop-breaking blocks (Section 6) form a fifth, cross-cutting category: they are simultaneously dynamic (or at minimum stateful across timesteps) and carry the `LoopBreaker` mixin, which gives them special treatment during graph traversal.

3 Dependency Resolution: Topological Sort via DFS

3.1 The Ordering Problem

For a forward evaluation pass to be correct, each block must be computed only after all blocks that feed into it have already produced their outputs for the current timestep. This is the classical topological ordering problem on a DAG.

In a pure feed-forward diagram with no cycles, standard post-order DFS from the sink nodes yields a valid execution order in $O(|V| + |E|)$ time. Feedback loops introduce directed cycles that make the graph no longer a DAG; these must be broken before topological sorting is possible.

3.2 Two-Pass Algorithm

EmbedSim resolves this in two passes implemented in `traverse_blocks_from_sinks_with_loops()`:

- **Pass 1 — Loop-Breaker Discovery.** A full DFS from every sink, visiting all reachable blocks, collects the set L of LoopBreaker instances. This pass is cycle-safe because it does not attempt to build an ordering; it simply records membership.
- **Pass 2 — Ordered DFS.** A second DFS builds the execution order. When traversing the input edges of a block, if an input block belongs to L it is inserted immediately (without recursing into its own inputs). This cuts every feedback edge, making the residual graph a DAG and allowing the DFS to complete without infinite recursion.

The algorithm is presented below in pseudocode:

```
function TRAVERSE(sinks):
    L ← ∅ // loop-breaker set
    visited ← ∅
    order ← []

    // Pass 1: find all LoopBreaker nodes
    for each sink s:
        FIND_BREAKERS(s, L, visited)

    visited ← ∅
    in_stack ← ∅ // cycle detection

    // Pass 2: post-order DFS producing execution order
    for each sink s:
        DFS(s, L, visited, in_stack, order)

    return order

function DFS(v, L, visited, in_stack, order):
    if v ∈ visited: return
    if v ∈ in_stack:
        raise AlgebraicLoopError(v) // true cycle, no breaker

    in_stack ← in_stack ∪ {v}

    for each u ∈ v.inputs:
        if u ∈ L:
            if u ∉ visited: // insert breaker in-place
                visited ← visited ∪ {u}
                order.append(u)
        else:
```

```

DFS(u, L, visited, in_stack, order)

in_stack ← in_stack \ {v}
visited ← visited ∪ {v}
order.append(v)           // post-order: v after all inputs

```

3.3 Complexity and Correctness

Time complexity: $O(|V| + |E|)$ for both passes combined. Each node is visited at most once per pass; each edge is examined at most once.

Space complexity: $O(|V|)$ for the visited and in_stack sets, plus $O(|V|)$ for the output list.

Correctness: The post-order property guarantees that for every non-loop-breaking edge (u, v) , u appears before v in order. Loop-breaking edges $(u \in L, v)$ are handled causally at runtime: u supplies its previous-timestep output via `get_loop_breaking_output()`, which is pre-populated before the main compute pass begins.

True algebraic loop detection: if DFS encounters a node already in in_stack and that node is not a LoopBreaker, the graph contains a pure algebraic loop — a cycle where every block requires the current output of another block in the same cycle. EmbedSim raises a `ValueError` naming the offending block, prompting the engineer to insert a `VectorDelay` in the path.

3.4 Post-Pass Fixup for Breaker Inputs

A subtlety arises when a LoopBreaker block itself has upstream dependencies that were skipped during Pass 2 (because DFS does not recurse into breaker inputs from the downstream side). EmbedSim performs a third, fixup pass: it iterates over every LoopBreaker already in order and checks whether its upstream inputs are present. Any missing input is added via a localised mini-DFS and inserted immediately before the LoopBreaker. The fixup iterates to a fixed point to handle chains of such missing nodes.

4 Numerical Integration of Dynamic States

4.1 Problem Statement

Let the complete system state be the concatenation of all dynamic block states: $X(t) = [x_1(t); x_2(t); \dots; x_n(t)] \in \mathbb{R}^N$. The coupled ODE system is:

$$\frac{dX}{dt} = F(X(t), U(t), t)$$

where $U(t)$ is the aggregate of all source block outputs at time t . EmbedSim solves this with fixed-step methods over the interval $[0, T]$ with step size dt .

4.2 Forward Euler (ODESolver.EULER)

Forward Euler is a first-order explicit method:

$$X(t + dt) = X(t) + dt \cdot F(X(t), U(t), t)$$

Global truncation error: $O(dt)$. Conditionally stable; the stability region for the test equation $dx/dt = \lambda x$ is $|1 + \lambda dt| \leq 1$, requiring $dt \leq 2/|\text{Re}(\lambda)|$ for stability. Appropriate for non-stiff systems where speed dominates accuracy requirements.

Implementation in EmbedSim (`_integrate_dynamics_euler`):

```
for each dynamic block b:
    u ← gather inputs from b.inputs
    b.derivative ← b.get_derivative(t, u)    // dx/dt = f(x,u,t)
    b.state ← b.state + dt · b.derivative    // Euler step
```

4.3 Runge-Kutta 4 (ODESolver.RK4)

RK4 is a fourth-order explicit method. For each dynamic block the four-stage computation is:

```
k1 = F( X(t),          U(t),          t          )
k2 = F( X(t)+dt/2·k1,  U(t+dt/2),    t+dt/2    )
k3 = F( X(t)+dt/2·k2,  U(t+dt/2),    t+dt/2    )
k4 = F( X(t)+dt·k3,    U(t+dt),       t+dt      )

X(t+dt) = X(t) + (dt/6) · (k1 + 2·k2 + 2·k3 + k4)
```

Global truncation error: $O(dt^4)$. This is four orders of magnitude more accurate than Euler for a given step size. RK4 is unconditionally the recommended solver for closed-loop control systems (e.g. PMSM FOC with SMC/SMO) where integrator accuracy directly affects controller convergence.

Implementation cost: RK4 calls `_compute_all_blocks()` three additional times per timestep (for the k2, k3, k4 intermediate stages). For a block graph with B blocks and S dynamic states, the per-step cost is $4 \times B$ compute calls versus $1 \times B$ for Euler. In practice this is acceptable because the larger step size afforded by RK4's superior accuracy often reduces the total number of steps required.

Intermediate state management: before the k1 stage, EmbedSim saves `initial_states[b] = b.state.copy()` for every dynamic block. Each subsequent stage perturbs `b.state` temporarily; the block graph is then re-evaluated to update $U(t + \delta)$ for that stage. After the k4 stage the final weighted combination is written back and `initial_states` is discarded.

4.4 Solver Selection Guidance

Scenario	Recommended Solver
Fast real-time preview, non-stiff plant	EULER
Closed-loop control, integrators present	RK4
PMSM FOC with SMC / SMO	RK4

Stiff ODE (eigenvalue ratio > 1000)	RK4 with reduced dt
LQR state-space, linear plant	RK4 or EULER
Pure discrete-time system (no integrators)	EULER

5 Dual-Backend Execution Architecture

5.1 Design Rationale

EmbedSim targets two distinct deployment environments that impose conflicting constraints. In simulation mode (Python), rapid iteration speed, NumPy broadcasting, and Matplotlib integration are paramount. In embedded mode (MCU), deterministic execution time, minimal RAM, and MISRA C:2012 compliance are mandatory. The dual-backend architecture resolves this tension by allowing the same block-diagram description to drive either runtime.

5.2 Backend Dispatcher

Every `VectorBlock.compute()` method is a thin dispatcher that routes to one of two implementations based on the instance flag `use_c_backend`:

```
def compute(self, t, dt, input_values):
    if self.use_c_backend:
        return self.compute_c(t, dt, input_values)
    return self.compute_py(t, dt, input_values)
```

Subclasses override `compute_py()` (required) and `compute_c()` (optional). The dispatcher is never overridden. This separation allows a developer to write and verify the Python path first, then introduce the C path incrementally, with the Python path always available as a reference.

5.3 Python Backend

The Python backend is the primary development path. It uses NumPy array operations throughout, giving vectorised performance without requiring compilation. All standard EmbedSim blocks (`VectorGain`, `VectorSum`, `StateSpaceBlock`, `VectorIntegrator`, etc.) ship with complete Python implementations. The Python backend is also the regression baseline: automated tests run the Python path and compare outputs against reference trajectories.

5.4 Cython / C Backend

The C backend enables zero-overhead execution of the hot simulation path. A Cython wrapper (`.pyx`) exposes the C function via a Python-compatible interface with the GIL released during computation. The wrapper receives inputs as a flat float64 array, calls the C function, and returns outputs as another flat float64 array. The `VectorBlock.compute_c()` method packs and unpacks these arrays.

A typical wrapper interaction:

```
def compute_c(self, t, dt, input_values):
    u = np.empty(N_INPUTS, dtype=np.float64)
    u[0] = input_values[0].value[0]    # I_alpha
    u[1] = input_values[0].value[1]    # I_beta
    u[2] = input_values[1].value[0]    # theta_r
    self._wrapper.set_inputs(u)
    self._wrapper.compute()             // GIL released
    y = self._wrapper.get_outputs()     // float64[N_OUT]
    self.output = VectorSignal(y, self.name)
    return self.output
```

Fallback: if the compiled extension is not present, EmbedSim catches the ImportError and transparently falls back to compute_py(). This means a simulation script runs correctly on any developer machine regardless of whether the Cython extensions have been built.

5.5 PYXInspector and Auto-Population

The PYXInspector module automates the boilerplate of declaring C code-generation metadata on block subclasses. When a subclass declares a PYX_FILE class attribute, the VectorBlock.__init_subclass__() hook fires and calls auto_populate_from_pyx(cls, pyx_file). PYXInspector parses the .pyx source and extracts:

- NUM_INPUTS — total scalar input count from the cdef function signature
- OUTPUT_SIZE — total scalar output count
- C_SOURCES — C source files required for compilation
- C_HEADERS — C header files to include

This lazy, declarative approach means that adding a new C-backed block requires only the PYX_FILE attribute; all metadata is derived automatically without manual synchronisation.

5.6 C Code Generation

The code_generator.py module (CodeGenStart / CodeGenEnd / LoopGenerator) traverses the topologically ordered block list and emits embedsim_loop.c and embedsim_loop.h into the project-root embedsim_gen/ directory. The emitter calls _emit_block() for each block in order, which in turn invokes PYXInspector on demand to resolve signal wiring from the C_INPUT_MAP dictionary. The generated .c file is a fixed-step, main-loop-friendly step function suitable for direct inclusion in an AURIX TriCore or Cortex-M4 bare-metal application.

6 Feedback Loop Breaking

6.1 The Algebraic Loop Problem

A feedback loop in a block diagram creates a directed cycle in G. If all blocks in the cycle are static (instantaneous), the cycle represents a true algebraic loop: the output of block A depends on B depends on A at the same instant. This is not generally solvable without Newton iteration and is almost always a modelling error in control-systems practice.

If at least one block in the cycle is causal (its output at time t depends only on its input at time t - dt), the cycle can be broken. This is the role of VectorDelay.

6.2 The LoopBreaker Interface

LoopBreaker is a Python mixin class. Any block that inherits from it declares to the simulation engine that it can supply a valid output for the current timestep without first needing its own inputs to be computed. The single required method is `get_loop_breaking_output()`, which returns the block's output from the previous timestep (or a user-supplied initial value at $t = 0$).

The engine uses LoopBreaker instances in two ways:

- During graph traversal (Pass 2 DFS): when an input edge points to a LoopBreaker, the edge is not followed recursively, cutting the cycle for ordering purposes.
- During `_compute_all_blocks()`: before the main forward pass, every LoopBreaker whose output is currently None is pre-seeded with `get_loop_breaking_output()`. Downstream blocks then see a non-None signal and proceed normally.

6.3 VectorDelay Implementation

VectorDelay is the canonical LoopBreaker. Its semantics are:

```
y(t)  = u(t - dt)           // one-step delay
y(0)  = initial             // user-specified initial condition
```

In code:

```
def compute_py(self, t, dt, input_values):
    // 1. Emit previous input
    val = self.last_output.value.copy() if self.last_output else zeros
    // 2. Store current input for next timestep
    self.last_output = VectorSignal(input_values[0].value.copy())
    self.output = VectorSignal(val, self.name)
    return self.output
```

The one-step delay introduces an approximation error of $O(dt)$ in the feedback path. For typical control-loop sample rates (10 kHz–20 kHz on AURIX), $dt \approx 50\text{--}50\mu\text{s}$; the phase lag introduced is ωdt radians, negligible at controller bandwidths well below the sample rate.

6.4 Topology Visualisation of Feedback Arcs

The TopologyPrinter renders feedback arcs as return paths drawn below the main block diagram. Each arc spans from the right edge of the driver block to the left edge(s) of the re-entry block(s), with the LoopBreaker name embedded in the arc label. This gives immediate visual confirmation of loop-breaking topology without running the simulation.

7 Signal Type System and Precision

7.1 Default Precision: float32

All VectorSignal objects default to `numpy.float32`. This mirrors the native word size of the target MCU (AURIX TriCore uses IEEE 754 single-precision in hardware). Using float32 throughout simulation

ensures that precision-related artefacts visible in hardware are faithfully reproduced in the simulation, giving higher fidelity than a float64-only environment.

The global default is controlled by the module-level constant `DEFAULT_DTYPE` in `core_blocks.py`. Changing this single constant switches the entire framework to float64 for applications where double precision is required (e.g. long-horizon trajectory integration, navigation algorithms).

7.2 Per-Block Precision Override

Any block may request a different precision by passing `dtype=np.float64` to its constructor. The dtype is propagated to all `VectorSignal` objects created by that block. Mixed-precision simulation is supported: a float64 integrator may be connected to a float32 gain block; NumPy handles the implicit cast at the array operation boundary.

7.3 Signal Dimension Conventions

EmbedSim uses 1-D NumPy arrays for all signals. Three-phase electrical quantities (V_abc, I_abc) are represented as shape-(3,) arrays. Clarke-transformed quantities (alpha-beta) use shape-(2,). Scalar signals use shape-(1,). This convention maps directly to C arrays in the generated embedsim_loop.c, where a 3-phase signal becomes a float32_t[3] local variable.

8 Simulation Lifecycle

8.1 Construction Phase

When `EmbedSim(sinks, T, dt, solver)` is called:

- `traverse_blocks_from_sinks_with_loops(sinks)` is called, producing the full ordered block list.
- Blocks are classified into `dynamic_blocks`, `static_blocks`, and `loop_breakers`.
- `SimulationStats` is initialised with `loop_breakers_count` and `feedback_loops_count`.
- `TopologyPrinter` is attached as `sim.topo` (lazy import to avoid circular dependencies).

8.2 Run Phase

EmbedSim.run() executes the following loop:

```

for each block b: b.reset()           // clear all outputs

t = 0
for step in range(int(T / dt)):
    _compute_all_blocks(t)           // forward pass in topo order
    scope.record(t)                  // sample monitored signals
    if solver == RK4:
        _integrate_dynamics_rk4(t)   // 4-stage state update
    else:
        _integrate_dynamics_euler(t) // 1-stage state update
    t += dt

_compute_all_blocks(T)                // capture final state
scope.record(T)

```

Note the ordering: outputs are computed and recorded before states are advanced. This implements the standard zero-order-hold (ZOH) discretisation convention: the output at step k reflects the state at the beginning of step k , not the end.

8.3 Signal Recording: VectorScope

VectorScope is a configurable oscilloscope. Before `run()`, the user calls `scope.add(block, indices, label)` to register blocks for recording. During each timestep `scope.record(t)` appends the current output values to per-label lists. After `run()`, `scope.get_signal(label, index)` returns a NumPy array of shape $(N,)$ for 1-D time-series analysis and plotting.

9 Architecture Summary

The following table summarises the key components and their roles:

Component	Role
VectorBlock	Abstract base; dual-backend dispatcher; DFS graph node
VectorSignal	Typed float32/64 vector; graph edge data carrier
traverse_blocks_from_sinks...	Two-pass DFS; topological sort; algebraic loop detection
LoopBreaker (mixin)	Marks causal delay blocks; cuts feedback cycles in DFS
VectorDelay	Canonical one-step delay; LoopBreaker implementation
EmbedSim	Orchestrator: construct, classify, run, record
VectorScope	Signal recorder; post-run NumPy array extraction
ODESolver	Enum: EULER ($O(dt)$), RK4 ($O(dt^4)$)
_integrate_dynamics_rk4	4-stage coupled RK4 across all dynamic blocks
CodeGenStart / LoopGenerator	C code emitter; produces <code>embedsim_loop.c</code> / <code>.h</code>
PYXInspector	Parses <code>.pyx</code> ; auto-populates <code>NUM_INPUTS</code> , <code>OUTPUT_SIZE</code> etc.
TopologyPrinter	Console + browser SVG block diagram renderer

10 References

- Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- Butcher, J. C. (2008). *Numerical Methods for Ordinary Differential Equations* (2nd ed.). Wiley.
- AUTOSAR Consortium. (2023). *AUTOSAR Classic Platform — Software Component Template*. Release 22-11.
- Infineon Technologies AG. (2022). *AURIX TC3xx User Manual*. Rev 2.0.

- MISRA Consortium. (2012). MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems.
- FMI Project. (2021). Functional Mock-up Interface Specification 2.0.4. Modelica Association.
- Cython Documentation. (2024). Cython 3.x Reference Manual. cython.readthedocs.io.
- Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–560.